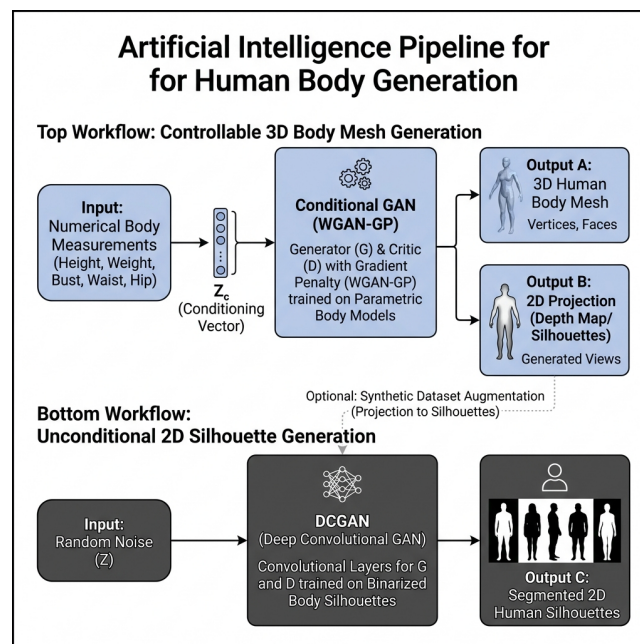


GAN Body Lab

Generación de Cuerpos Humanos mediante
Redes Generativas Adversariales



Autores: Santiago Álvarez Geanta
Alejandro García Belmonte
Guillermo García Blázquez
Taron Sargsyan

Asignatura: Redes Neuronales y Aprendizaje Profundo

Institución: Universidad de Alicante · Ingeniería en Inteligencia Artificial

Fecha: Mayo 2026

Índice

1. Introducción y Motivación	3
2. Fundamentos Teóricos de las GANs	3
2.1. Contexto: Modelos Generativos	3
2.2. El Framework GAN Original	4
2.3. La Función Objetivo Minimax	4
2.4. La Distribución Normal como Espacio Latente	4
2.5. El Equilibrio de Nash y la Convergencia	5
2.6. Algoritmo de Entrenamiento	6
3. Variantes y Mejoras de las GANs	7
3.1. DCGAN: Deep Convolutional GAN	7
3.2. Conditional GAN (cGAN)	7
3.3. WGAN y WGAN-GP	8
3.3.1. Problemas del GAN original	8
3.3.2. WGAN-GP (Gulrajani et al., 2017)	8
4. Descripción del Proyecto	8
4.1. Objetivos	8
4.2. Pipeline General	9
5. Datasets	9
5.1. NOMO3D: Medidas Antropométricas	9
5.2. SMPL: Modelo Paramétrico de Cuerpo Humano	10
5.3. TNT15: Dataset de Siluetas	10
6. Arquitectura del Sistema	10
6.1. Pipeline Tabular MLP: Medidas → Cuerpo SMPL	10
6.2. Pipeline de Imagen: Ruido → Silueta Humana	10
6.3. Pipeline Tab Transformer: Nube de Puntos → Cuerpo SMPL	11
7. Modelos Implementados	11
7.1. Generador Tabular Condicional	11
7.2. Discriminador Tabular	12
7.3. Generador de Imagen (DCGAN)	12
7.4. Discriminador de Imagen (Crítico CNN)	12
8. Pre-procesamiento: Beta Fitter (Gauss-Newton)	13
9. Entrenamiento WGAN-GP	14
9.1. Hiperparámetros	14
9.2. Loop de Entrenamiento Tabular	14
9.3. Gradient Penalty: Implementación	15

10. Inferencia y Resultados	15
10.1. Inferencia Tabular (2D y 3D)	15
10.2. Salida 2D — Proyección Ortográfica	16
10.3. Salida 3D — Malla SMPL	16
10.4. Grid de Imágenes TNT15	17
11. Evaluación de los Modelos	17
11.1. Evaluación del Pipeline MLP Tabular (MAE)	17
11.2. Evaluación del Tab Transformer (FID e IS)	18
12. Reflexión: Estabilidad y Modos de Fallo	18
12.1. Mode Collapse	18
12.2. Gradientes Desaparecidos	18
12.3. Normalización de Medidas	18
13. Estructura del Repositorio	18
14. Pipeline Tab Transformer: Nubes de Puntos → Cuerpo SMPL	19
14.1. Motivación: ¿Por qué un Transformer sobre nubes de puntos?	19
14.2. Dataset de Nubes de Puntos	20
14.2.1. Normalización de las nubes de puntos	20
14.3. Generador Tab Transformer	21
14.4. Discriminador Tab Transformer	22
14.5. Hiperparámetros del Tab Transformer	23
14.6. Bucle de Entrenamiento	24
14.7. Evaluación: FID e Inception Score Tabulares	24
14.7.1. FID Tabular (Fréchet Inception Distance)	24
14.7.2. Inception Score Tabular	25
14.8. Resultados Experimentales	25
14.8.1. Análisis del FID	26
14.8.2. Análisis del Inception Score	26
14.8.3. Interpretación y comparación con el MLP	26
14.9. Inferencia del Tab Transformer	27
14.10 Integración en la CLI	27
14.11 Lo que deliberadamente no hace este diseño	27
15. Conclusiones	28

1. Introducción y Motivación

El presente proyecto, denominado **GAN Body Lab**, aborda el problema de la generación sintética de cuerpos humanos a partir de datos antropométricos. Esta tarea tiene aplicaciones directas en industrias como la moda, el diseño ergonómico, la medicina y los videojuegos, donde disponer de avatares corporales realistas y controlables supone una ventaja significativa.

La motivación central surge de una pregunta práctica: ¿es posible construir un modelo que, recibiendo únicamente un conjunto reducido de medidas corporales (altura, contorno de pecho, cintura, cadera, etc.), sea capaz de sintetizar una forma corporal 3D plausible y coherente con esas restricciones? La respuesta que este proyecto propone es afirmativa, y se apoya en el uso de **Redes Generativas Adversariales** (GANs) condicionadas por las medidas de entrada.

El proyecto implementa tres pipelines complementarios e independientes:

- (1) **Pipeline Tabular (MLP)**: aprende una correspondencia generativa entre medidas antropométricas y parámetros de forma del modelo SMPL (*betas*). La salida puede exportarse como imagen 2D (PNG) o malla 3D (OBJ).
- (2) **Pipeline de Imagen**: aprende la distribución visual de imágenes de cuerpos humanos segmentados del dataset TNT15, generando siluetas plausibles desde ruido aleatorio, sin condicionamiento explícito.
- (3) **Pipeline Tab Transformer**: una rama experimental que explora el uso de un Transformer sobre nubes de puntos 3D como condición, sirviendo de línea base negativa para demostrar que la representación del condicionamiento importa tanto como la arquitectura.

Los tres pipelines comparten la misma arquitectura de pérdida, **WGAN-GP** (Wasserstein GAN con Gradient Penalty) [1], que es una de las variantes más estables del framework GAN original propuesto por Goodfellow et al. en 2014 [2].

2. Fundamentos Teóricos de las GANs

2.1. Contexto: Modelos Generativos

Los modelos generativos aprenden a estimar la distribución subyacente $p(\mathbf{x})$ de los datos de entrenamiento con el fin de generar nuevas muestras realistas. Los tres paradigmas principales son:

VAE	Codifica datos en un espacio latente estructurado y los reconstruye. Entrenamiento estable, pero produce muestras desenfocadas.
GAN	Enfrenta un generador contra un discriminador en un juego adversarial. Alta calidad visual, pero inestable y propenso al <i>mode collapse</i> .
Diffusion	Revierte un proceso gradual de adición de ruido. Máxima calidad y diversidad, pero inferencia lenta (muchos pasos de denoising).

El presente proyecto opta por GANs porque las salidas de alta nitidez son prioritarias, y la comunidad ha desarrollado variantes estables (WGAN-GP) que hacen el entrenamiento manejable.

2.2. El Framework GAN Original

Las Redes Generativas Adversariales fueron introducidas por Goodfellow et al. [2] en el artículo seminal “*Generative Adversarial Nets*” (NIPS 2014). El framework propone entrenar simultáneamente dos redes:

- **Generador G :** aprende una distribución p_g sobre los datos. Recibe ruido latente $\mathbf{z} \sim p_z(\mathbf{z})$ y lo mapea al espacio de datos mediante $G(\mathbf{z}; \theta_g)$. Su objetivo es que sus muestras sean indistinguibles de los datos reales.
- **Discriminador D :** recibe muestras reales $\mathbf{x} \sim p_{\text{data}}(\mathbf{x})$ y falsas $G(\mathbf{z})$, y produce un escalar $D(\mathbf{x}; \theta_d)$ que estima la probabilidad de que $\mathbf{x}$ provenga de los datos reales. Su objetivo es clasificar correctamente reales de falsos.

La metáfora de los autores es ilustrativa: G actúa como falsificadores que producen moneda falsa, mientras que D actúa como la policía que intenta detectarlos. La competición entre ambos los impulsa a mejorar mutuamente.

2.3. La Función Objetivo Minimax

El entrenamiento conjunto de G y D corresponde a un juego minimax de dos jugadores con la función de valor $V(G, D)$:

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))] \quad (1)$$

Interpretación de cada término.

- $\mathbb{E}[\log D(\mathbf{x})]$: D recibe una muestra *real* $\mathbf{x}$ y quiere que $D(\mathbf{x}) \rightarrow 1$, es decir, $\log D(\mathbf{x}) \rightarrow 0$ (máximo). Si D es engañado y produce 0.5, entonces $\log(0.5) = -0.69$, lo cual es peor para él.
- $\mathbb{E}[\log(1 - D(G(\mathbf{z})))]$: G genera una muestra falsa. D quiere $D(G(\mathbf{z})) \rightarrow 0$; G quiere lo contrario: $D(G(\mathbf{z})) \rightarrow 1$, haciendo este término muy negativo (mínimo).

En la práctica, $\log(1 - D(G(\mathbf{z})))$ satura cuando D rechaza con fiabilidad las muestras al inicio. Por eso, en la práctica se entrena G para *maximizar* $\log D(G(\mathbf{z}))$: el punto fijo es el mismo, pero los gradientes son más fuertes al inicio.

2.4. La Distribución Normal como Espacio Latente

El ruido latente $\mathbf{z}$ se muestrea de $\mathcal{N}(\mathbf{0}, \mathbf{I})$. Esta elección no es arbitraria:

- La distribución normal cubre todo $\mathbb{R}^n$ sin “zonas muertas”, eliminando regiones del espacio latente que nunca se explorarían.

- Cualquier distribución puede obtenerse transformando una normal mediante una función suficientemente expresiva (y una red profunda lo es).
- $\mathcal{N}(\mathbf{0}, \mathbf{I})$ es *isotrópica*: todas las direcciones latentes son tratadas por igual, lo que hace funcionar la interpolación y la aritmética de vectores en el espacio latente.

En el presente proyecto, el generador tabular usa `noise_dim = 64` y el generador de imagen usa `noise_dim = 128`.

2.5. El Equilibrio de Nash y la Convergencia

Desde la teoría de juegos, el entrenamiento busca el **equilibrio de Nash**: ningún jugador puede mejorar cambiando únicamente su propia estrategia. Esto ocurre cuando $D(G(\mathbf{z})) = 0.5$: el discriminador esencialmente lanza una moneda, porque las muestras falsas son indistinguibles de las reales.

Goodfellow et al. demostraron formalmente:

Proposición 1 (Discriminador Óptimo)

Para G fijo, el discriminador óptimo D^* es:

$$D_G^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})} \quad (2)$$

Teorema 1 (Óptimo Global)

El mínimo global del criterio de entrenamiento $C(G)$ se alcanza si y solo si $p_g = p_{\text{data}}$. En ese punto, $C(G) = -\log 4$.

La función de valor del minimax puede reformularse en términos de la **divergencia Jensen-Shannon**:

$$C(G) = -\log 4 + 2 \cdot \text{JSD}(p_{\text{data}} \| p_g) \quad (3)$$

Dado que JSD es siempre no-negativa y vale cero solo cuando las distribuciones son iguales, el único mínimo global es $p_g = p_{\text{data}}$.

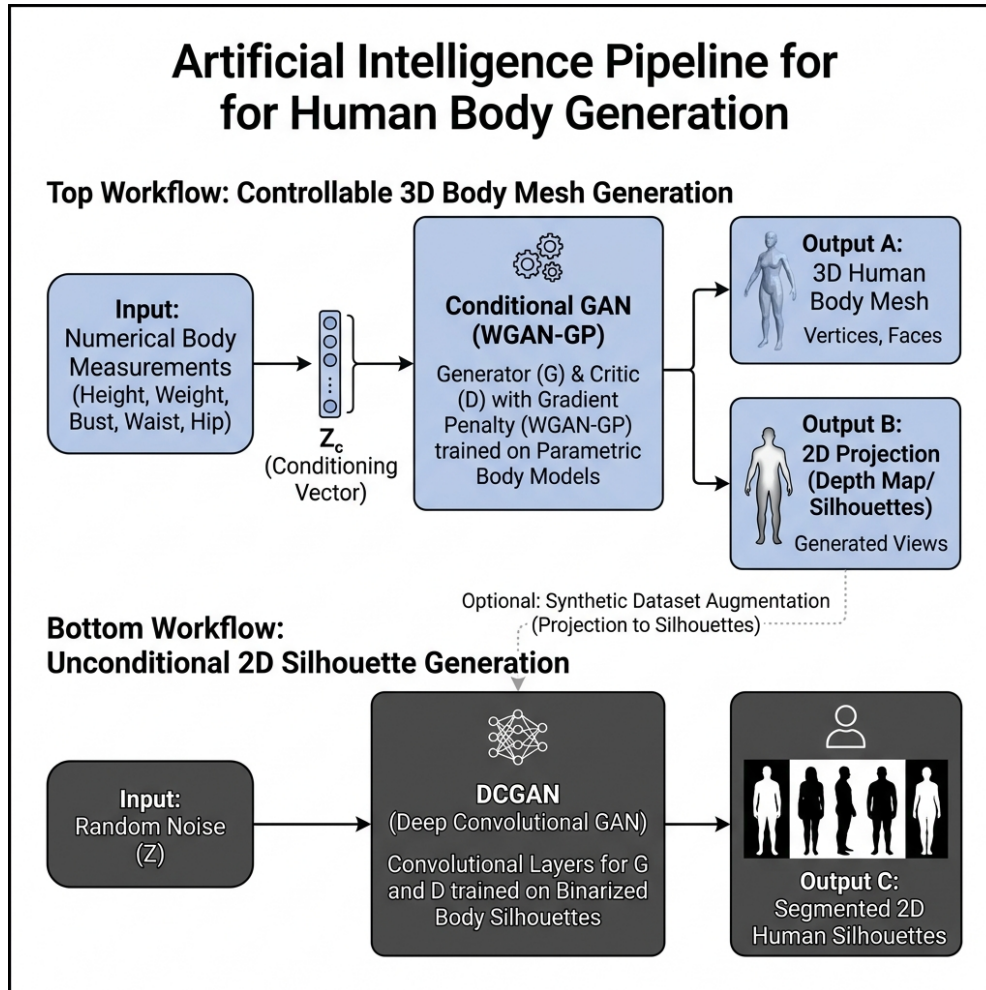


Figura 1: Pipeline completo del sistema GAN Body Lab. **Arriba:** workflow de generación de malla 3D condicionada por medidas (Conditional WGAN-GP + SMPL). **Abajo:** workflow de generación de siluetas 2D incondicional (DCGAN + WGAN-GP sobre TNT15).

2.6. Algoritmo de Entrenamiento

El Algoritmo 1 es el algoritmo de entrenamiento minimax por lotes estocásticos presentado en el paper original.

Algorithm 1 Entrenamiento minibatch estocástico de GANs (Goodfellow et al., 2014)

-
- ```

1: for número de iteraciones de entrenamiento do
2: for k pasos do
3: Muestrear $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\} \sim p_z(\mathbf{z})$
4: Muestrear $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\} \sim p_{\text{data}}(\mathbf{x})$
5: Actualizar D ascendiendo su gradiente estocástico:

```

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m [\log D(\mathbf{x}^{(i)}) + \log(1 - D(G(\mathbf{z}^{(i)})))]$$

- ```

6:   end for
7:   Muestrear  $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\} \sim p_z(\mathbf{z})$ 
8:   Actualizar  $G$  descendiendo su gradiente estocástico:

```

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(\mathbf{z}^{(i)})))$$

- ```

9: end for

```
- 

### 3. Variantes y Mejoras de las GANs

#### 3.1. DCGAN: Deep Convolutional GAN

Propuesto por Radford et al. [3], DCGAN introduce las siguientes guías de arquitectura para GANs convolucionales estables:

- Reemplazar el *pooling* con convoluciones con *stride* (discriminador) y convoluciones fraccionadas (*generator*).
- Usar *BatchNorm* tanto en el generador como en el discriminador.
- Eliminar las capas completamente conectadas en arquitecturas profundas.
- Usar ReLU en el generador (excepto la salida, que usa Tanh).
- Usar LeakyReLU en el discriminador en todas las capas.

#### 3.2. Conditional GAN (cGAN)

Propuesta por Mirza y Osindero [4] en “Conditional Generative Adversarial Nets” (2014), la cGAN extiende el framework GAN añadiendo información de condicionamiento  $\mathbf{y}$  a ambas redes. La función objetivo condicionada es:

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x} \mid \mathbf{y})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z} \mid \mathbf{y})))] \quad (4)$$

En el generador, el ruido  $p_z(\mathbf{z})$  y  $\mathbf{y}$  se combinan en una representación oculta conjunta. En el discriminador,  $\mathbf{x}$  e  $\mathbf{y}$  se presentan como entradas adicionales. En el presente proyecto,



el Generador tabular es exactamente una cGAN: recibe tanto  $\mathbf{z} \in \mathbb{R}^{64}$  como las 10 medidas corporales normalizadas, y genera los 10 parámetros  $\beta$  SMPL condicionados a esas medidas.

### 3.3. WGAN y WGAN-GP

#### 3.3.1. Problemas del GAN original

El GAN original sufre dos problemas principales:

1. **Gradientes desaparecidos:** cuando  $D$  es demasiado bueno al inicio,  $\log(1 - D(G(\mathbf{z}))) \approx 0$  y el gradiente del generador se desvanece.
2. **Mode collapse:**  $G$  aprende a producir siempre el mismo output, ignorando la diversidad real del espacio de datos.

#### 3.3.2. WGAN-GP (Gulrajani et al., 2017)

WGAN-GP [1] reemplaza la función de valor por la distancia de Wasserstein e impone la restricción 1-Lipschitz mediante una **penalización del gradiente**:

$$\mathcal{L}_D = -\mathbb{E}[D(\mathbf{x}_{\text{real}})] + \mathbb{E}[D(G(\mathbf{z}))] + \lambda \mathbb{E}[(\|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}})\|_2 - 1)^2] \quad (5)$$

$$\mathcal{L}_G = -\mathbb{E}[D(G(\mathbf{z}))] \quad (6)$$

donde  $\hat{\mathbf{x}} = \alpha \mathbf{x}_{\text{real}} + (1 - \alpha) G(\mathbf{z})$ , con  $\alpha \sim \mathcal{U}[0, 1]$ , y  $\lambda = 10$  (usado en este proyecto como `lambda_gp=10.0`).

#### Nota técnica: BatchNorm $\times$ WGAN-GP

La penalización del gradiente requiere derivar la salida del discriminador respecto a cada muestra individualmente. BatchNorm acopla las muestras del batch, lo que rompe ese supuesto. Por eso, en todos los discriminadores de este proyecto se usa **GroupNorm(num\_groups=1)** o **LayerNorm** (equivalentes cuando se normaliza por muestra) en lugar de BatchNorm.

## 4. Descripción del Proyecto

### 4.1. Objetivos

1. Implementar una Conditional WGAN-GP que mapee 10 medidas antropométricas a 10 parámetros de forma SMPL ( $\beta$ ).
2. Implementar un sistema de ajuste de  $\beta$  SMPL (pseudo-ground-truth) mediante optimización Gauss-Newton no lineal.
3. Implementar un DCGAN + WGAN-GP capaz de generar siluetas humanas plausibles desde ruido aleatorio usando TNT15.

4. Implementar un Tab Transformer condicionado por nubes de puntos 3D como línea base experimental, y evaluar con métricas generativas (FID e Inception Score).
5. Proveer una CLI completa (`main.py`) para entrenar, evaluar e inferir con los tres pipelines.

## 4.2. Pipeline General

El sistema completo, ilustrado en la Figura 1, se divide en tres workflows:

**Workflow 1 — MLP tabular** (Generación de malla 3D controlable): Recibe medidas numéricas corporales combinadas con un vector de condicionamiento  $Z_c$ . Entra en una Conditional WGAN-GP y produce dos salidas: (A) malla 3D del cuerpo humano (vértices/caras, OBJ) y (B) proyección 2D (mapa de profundidad/siluetas, PNG).

**Workflow 2 — DCGAN de imagen** (Generación incondicional de siluetas 2D): Recibe ruido aleatorio  $Z$  que entra en una DCGAN entrenada sobre siluetas binarizadas del dataset TNT15, produciendo siluetas 2D segmentadas.

**Workflow 3 — Tab Transformer** (Línea base experimental): Recibe una nube de 256 puntos 3D submuestreados de SMPL junto con ruido latente  $Z$ . Un Transformer procesa los tokens geométricos y genera los 10 parámetros  $\beta$  SMPL. Sirve para comparar el efecto de la representación del condicionamiento frente al MLP.

## 5. Datasets

### 5.1. NOMO3D: Medidas Antropométricas

NOMO3D es el dataset principal del pipeline tabular. Contiene medidas corporales de sujetos humanos reales en formato textual (archivos `.txt` con líneas `MEASURE nombre=valor` en centímetros). El sistema lee 10 medidas para construir el vector de condicionamiento:

Cuadro 1: Medidas corporales usadas como vector de condicionamiento.

| Campo NOMO3D         | Significado físico                 |
|----------------------|------------------------------------|
| Head_Top_Height      | Altura total del cuerpo (cm)       |
| BUST_Circ            | Contorno de pecho/busto (cm)       |
| NaturalWAIST_Circ    | Contorno de cintura natural (cm)   |
| HIP_Circ             | Contorno de cadera (cm)            |
| NeckBase_Circ        | Contorno de cuello en la base (cm) |
| Shoulder_to_Shoulder | Anchura de hombros (cm)            |
| Inseam               | Tiro interior (cm)                 |
| Outseam              | Largo exterior de pierna (cm)      |
| Thigh_Circ           | Contorno de muslo (cm)             |
| Bicep_Circ           | Contorno de bíceps (cm)            |

La división train/test es 80 %/20 % (configurable), con semilla fija (`np.random.seed(42)`) para reproducibilidad. Las medidas se normalizan con Z-score calculado sobre el conjunto de entrenamiento y guardado en `internal/experiments/meas_scaler.npz`.

## 5.2. SMPL: Modelo Paramétrico de Cuerpo Humano

SMPL [5] (Skinned Multi-Person Linear Model) es el modelo paramétrico de cuerpo humano que sirve de “capa de decodificación” para convertir los  $\beta$  generados por la GAN en una malla 3D coherente. SMPL describe la forma corporal mediante:

- **Parámetros de forma**  $\beta \in \mathbb{R}^{10}$ : captura variaciones de forma entre individuos (talla, complexión, proporciones).
- **Parámetros de pose**  $\theta$ : ángulos articulares.
- La malla resultante tiene **6890 vértices** y  $\approx 13000$  caras.

## 5.3. TNT15: Dataset de Siluetas

TNT15 contiene cuatro sujetos (`mr`, `pz`, `sg`, `sp`) grabados desde cinco cámaras (00–04), con PNGs ya segmentados (sujeto en gris sobre fondo negro).

- $\approx 100\,000$  frames brutos;  $\approx 20\,600$  efectivos tras submuestreo con `frame_stride=5`.
- Imágenes en escala de grises (1 canal).
- Redimensionado a  $64 \times 64$ , normalización a  $[-1, 1]$ .
- Split train/test: 90 %/10 % por orden alfabético.

# 6. Arquitectura del Sistema

## 6.1. Pipeline Tabular MLP: Medidas $\rightarrow$ Cuerpo SMPL

El pipeline tabular es una Conditional GAN donde generador y discriminador son MLP:

Entradas del generador:  $\mathbf{z} \in \mathbb{R}^{64}$ ,  $\mathbf{c} \in \mathbb{R}^{10}$  (medidas norm.)  
 Salida del generador:  $\beta \in \mathbb{R}^{10}$  (shape params SMPL)  
 Entradas del discriminador:  $\beta \in \mathbb{R}^{10}$ ,  $\mathbf{c} \in \mathbb{R}^{10}$   
 Salida del discriminador:  $s \in \mathbb{R}$  (score Wasserstein, sin activación)

## 6.2. Pipeline de Imagen: Ruido $\rightarrow$ Silueta Humana

El pipeline de imagen sigue DCGAN + WGAN-GP sin condicionamiento:

Entrada del generador:  $\mathbf{z} \in \mathbb{R}^{128}$   
 Salida del generador:  $\mathbf{I} \in \mathbb{R}^{1 \times 64 \times 64}$  (imagen en  $[-1, 1]$ )  
 Entrada del discriminador:  $\mathbf{I} \in \mathbb{R}^{1 \times 64 \times 64}$   
 Salida del discriminador:  $s \in \mathbb{R}$  (sin sigmoid)

### 6.3. Pipeline Tab Transformer: Nube de Puntos $\rightarrow$ Cuerpo SMPL

El pipeline Tab Transformer es una Conditional GAN donde generador y discriminador son Transformers que procesan nubes de puntos 3D como tokens:

Entradas del generador:  $\mathbf{z} \in \mathbb{R}^{64}$ ,  $\mathbf{P} \in \mathbb{R}^{256 \times 3}$  (nube de puntos norm.)  
 Salida del generador:  $\beta \in \mathbb{R}^{10}$  (shape params SMPL)  
 Entradas del discriminador:  $\beta \in \mathbb{R}^{10}$ ,  $\mathbf{P} \in \mathbb{R}^{256 \times 3}$   
 Salida del discriminador:  $s \in \mathbb{R}$  (score Wasserstein, sin activación)

La arquitectura detallada de este pipeline se describe en la Sección 14.

## 7. Modelos Implementados

### 7.1. Generador Tabular Condicional

**Archivo:** `src/models/generator.py`

El generador implementado es `_GeneratorWithResidual`, que divide la red en tres segmentos:

**Pre-peak.** Desde la entrada concatenada  $[\mathbf{z}; \mathbf{c}] \in \mathbb{R}^{74}$  hasta el pico de dimensionalidad:

`Linear(74→256) + LayerNorm + LeakyReLU(0.2)`  
`Linear(256→512) + LayerNorm + LeakyReLU(0.2)`  
`Linear(512→1024) + LayerNorm + LeakyReLU(0.2)`

**Bloque residual** en el pico (dim = 1024):

$\text{ResBlock}(\mathbf{x}) = \text{LeakyReLU}(\mathbf{x} + [\text{Lin} \rightarrow \text{LN} \rightarrow \text{LReLU} \rightarrow \text{Lin} \rightarrow \text{LN}](\mathbf{x}))$

**Post-peak** hasta la salida:

`Linear(1024→512) + LayerNorm + LeakyReLU(0.2)`  
`Linear(512→256) + LayerNorm + LeakyReLU(0.2)`  
`Linear(256→10) (sin activación)`

Inicialización: Kaiming Normal con  $a = 0.2$ ; sesgo=0.

## 7.2. Discriminador Tabular

**Archivo:** `src/models/discriminator.py`

$$[\beta; \mathbf{c}] \in \mathbb{R}^{20} \xrightarrow{\text{Lin+LReLU}} 256 \xrightarrow{\text{Lin+LReLU}} 128 \xrightarrow{\text{Lin+LReLU}} 64 \xrightarrow{\text{Lin}} \mathbb{R} \text{ (sin sigmoid)}$$

Sin BatchNorm: la penalización del gradiente WGAN-GP requiere independencia entre muestras, que BatchNorm viola.

## 7.3. Generador de Imagen (DCGAN)

**Archivo:** `src/models/img_generator.py`

Arquitectura para  $64 \times 64$ , 1 canal, `noise_dim=128`, `g_feature_maps=64`:

Cuadro 2: Arquitectura del ImgGenerator.

| Capa         | Operación          | Tamaño salida      | Norm.     |
|--------------|--------------------|--------------------|-----------|
| Entrada      | reshape            | $(B, 128, 1, 1)$   | —         |
| ConvT1       | $4 \times 4$ , s=1 | $(B, 512, 4, 4)$   | BN + ReLU |
| ConvT2       | $4 \times 4$ , s=2 | $(B, 256, 8, 8)$   | BN + ReLU |
| ConvT3       | $4 \times 4$ , s=2 | $(B, 128, 16, 16)$ | BN + ReLU |
| ConvT4       | $4 \times 4$ , s=2 | $(B, 64, 32, 32)$  | BN + ReLU |
| ConvT5 (out) | $4 \times 4$ , s=2 | $(B, 1, 64, 64)$   | Tanh      |

El número de bloques se calcula automáticamente como  $n = \log_2(\text{image\_size}) - 2$ , haciendo la arquitectura independiente del tamaño objetivo.

## 7.4. Discriminador de Imagen (Crítico CNN)

**Archivo:** `src/models/img_discriminator.py`

Cuadro 3: Arquitectura del ImgDiscriminator.

| Capa        | Operación          | Tamaño salida      | Norm.       |
|-------------|--------------------|--------------------|-------------|
| Entrada     | —                  | $(B, 1, 64, 64)$   | —           |
| Conv1       | $4 \times 4$ , s=2 | $(B, 64, 32, 32)$  | LeakyReLU   |
| Conv2       | $4 \times 4$ , s=2 | $(B, 128, 16, 16)$ | GN(1)+LReLU |
| Conv3       | $4 \times 4$ , s=2 | $(B, 256, 8, 8)$   | GN(1)+LReLU |
| Conv4       | $4 \times 4$ , s=2 | $(B, 512, 4, 4)$   | GN(1)+LReLU |
| Conv5 (out) | $4 \times 4$ , s=1 | $(B, 1, 1, 1)$     | —           |
| flatten     | —                  | $(B, 1)$           | sin sigmoid |

`GroupNorm(num_groups=1)`: equivalente a `LayerNorm` sobre  $(C, H, W)$ . Normaliza por muestra de forma independiente, respetando el supuesto de independencia requerido por WGAN-GP.

## 8. Pre-procesamiento: Beta Fitter (Gauss-Newton)

**Archivo:** `src/data/beta_fitter.py`

Antes de entrenar la GAN tabular, se obtiene para cada muestra de NOMO3D el vector  $\beta^*$  que mejor reproduce las 10 medidas del dataset. La cadena de medición (MeasureBody sobre trimesh) no es diferenciable, por lo que se usa diferencias finitas centrales para el Jacobiano.

---

### Algorithm 2 Ajuste de $\beta$ SMPL por Gauss-Newton

---

**Require:** Target de medidas  $\mathbf{t} \in \mathbb{R}^{10}$ , género, tolerancias

**Ensure:**  $\beta^* \in [-3, 3]^{10}$

```

1: Fase 1 — Warm-start lineal (linearización en $\beta = \mathbf{0}$):
2: $\mathbf{m}_0 \leftarrow \text{SMPL}(\mathbf{0}, \text{gender})$
3: for $k = 0, \dots, 9$ do
4: $J_{:,k} \leftarrow [\text{SMPL}(\mathbf{0} + \varepsilon \mathbf{e}_k) - \text{SMPL}(\mathbf{0} - \varepsilon \mathbf{e}_k)] / (2\varepsilon)$
5: end for
6: $\beta_0 \leftarrow \text{clip}(\text{lstsq}(J, \mathbf{t} - \mathbf{m}_0), -3, 3)$
7: Fase 2 — Gauss-Newton iterativo (re-linearización en β_k):
8: for $\text{iter} = 1, \dots, 8$ do
9: Re-computar J_k en β_k (idem, costo $2 \cdot 10 + 1$ eval.)
10: $\Delta\beta \leftarrow \text{lstsq}(J_k, \mathbf{t} - \text{SMPL}(\beta_k))$
11: Backtracking: halvar step hasta $\|r_{\text{trial}}\| \leq \|r_k\|$
12: $\beta_{k+1} \leftarrow \text{clip}(\beta_k + \text{step} \cdot \Delta\beta, -3, 3)$
13: if $\|\Delta\beta\| < 10^{-3}$ o mejora relativa $< 10^{-4}$ then
14: break
15: end if
16: end for
17: return β_k

```

---

Los  $\beta$  ajustados se guardan en `internal/data/betas_cache/{id}.npy`; las estadísticas de convergencia en `internal/logs/beta_fit_stats.csv`.

## 9. Entrenamiento WGAN-GP

### 9.1. Hiperparámetros

Cuadro 4: Hiperparámetros de entrenamiento para ambos pipelines.

| Hiperparámetro     | Tabular (HParams)      | Imagen (ImgHParams) |
|--------------------|------------------------|---------------------|
| noise_dim          | 64                     | 128                 |
| batch_size         | 128                    | 64                  |
| epochs             | 3 000                  | 100                 |
| lr_g = lr_d        | $10^{-4}$              | $10^{-4}$           |
| $\beta_1, \beta_2$ | 0.5, 0.9               | 0.5, 0.9            |
| n_critic           | 5                      | 5                   |
| lambda_gp          | 10.0                   | 10.0                |
| g_hidden_dims      | [256,512,1024,512,256] | —                   |
| image_size         | —                      | 64                  |
| channels           | —                      | 1 (grayscale)       |
| frame_stride       | —                      | 5                   |
| train_split        | 0.8                    | 0.9                 |

**Justificación de  $\beta_1 = 0.5, \beta_2 = 0.9$ .** Los valores por defecto de Adam (0.9, 0.999) introducen demasiada inercia para WGAN-GP. Con  $\beta_1 = 0.5$  se reduce el momentum y el entrenamiento es más estable, tal como recomienda el paper de Gulrajani.

**Justificación de n\_critic=5.** El discriminador se actualiza 5 veces por cada actualización del generador. Esto da al crítico tiempo de adaptarse antes de que  $G$  se mueva, lo que es crucial para que la estimación de la distancia de Wasserstein sea fiable.

### 9.2. Loop de Entrenamiento Tabular

Por cada batch ( $c, \beta_{\text{real}}$ ) del dataset NOMO3D:

```

1 # Normalizar medidas
2 meas_norm = (meas - meas_mean) / meas_std
3
4 # --- n_critic=5 pasos del Discriminador ---
5 for _ in range(n_critic):
6 z = torch.randn(batch, noise_dim)
7 beta_fake = G(z, meas_norm).detach()
8 d_real = D(beta_real, meas_norm)
9 d_fake = D(beta_fake, meas_norm)
10 gp = compute_gradient_penalty(beta_real, beta_fake, meas_norm)
11 L_D = -d_real.mean() + d_fake.mean() + lambda_gp * gp
12 L_D.backward(); opt_D.step()
13
14 # --- 1 paso del Generador ---
15 z = torch.randn(batch, noise_dim)

```

```

16 beta_fake = G(z, meas_norm)
17 L_G = -D(beta_fake, meas_norm).mean()
18 L_G.backward(); opt_G.step()

```

Listing 1: Loop WGAN-GP tabular (extracto de train.py)

### 9.3. Gradient Penalty: Implementación

```

1 def compute_gradient_penalty(real, fake, cond):
2 # Interpolador entre muestras reales y falsas
3 alpha = torch.rand(real.size(0), 1, device=device)
4 x_hat = (alpha * real + (1-alpha) * fake).requires_grad_(True)
5 d_hat = D(x_hat, cond)
6 # Calcular gradiente respecto a la interpolacion
7 grads = autograd.grad(outputs=d_hat, inputs=x_hat,
8 grad_outputs=torch.ones_like(d_hat),
9 create_graph=True)[0]
10 grads = grads.view(grads.size(0), -1)
11 gp = ((grads.norm(2, dim=1) - 1) ** 2).mean()
12 return gp

```

Listing 2: compute\_gradient\_penalty (train.py)

Para el discriminador de imagen, `alpha` tiene forma  $(B, 1, 1, 1)$  en lugar de  $(B, 1)$ , para interpolar en el espacio de imagen  $\mathbb{R}^{1 \times 64 \times 64}$ .

## 10. Inferencia y Resultados

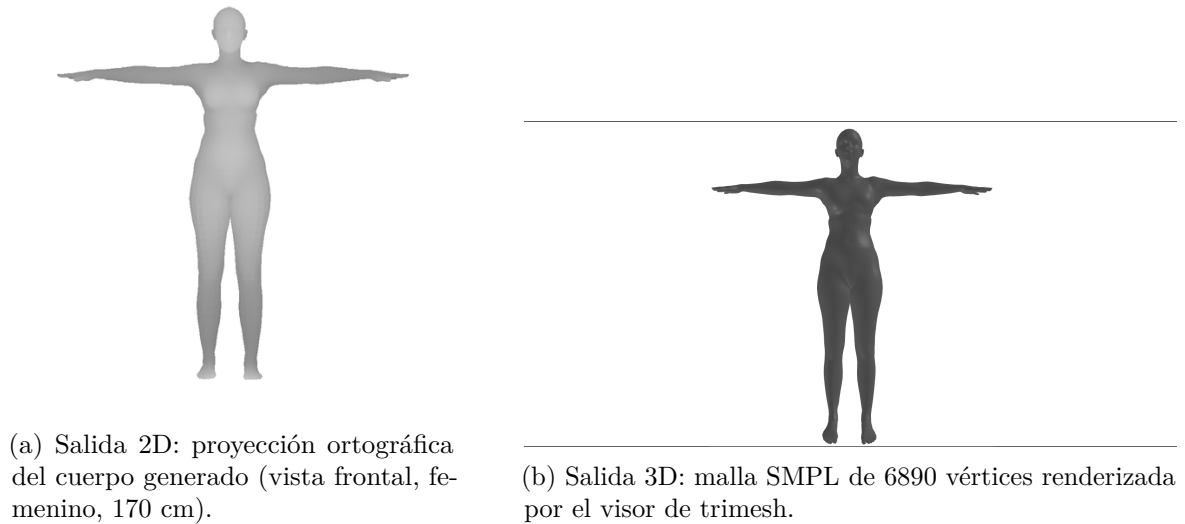
### 10.1. Inferencia Tabular (2D y 3D)

El proceso de inferencia tabular sigue estos pasos:

1. Cargar el último checkpoint `wgangp_ckpt_*.pt` (por mtime).
2. Cargar el scaler (`meas_scaler.npz`) del entrenamiento.
3. Normalizar las 10 medidas de entrada.
4. Generar  $N = 32$  muestras  $z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  en paralelo.
5. Pasar todas por  $G$  y promediar:  $\beta^* = \frac{1}{N} \sum_{i=1}^N G(z_i, c)$ .
6. Instanciar SMPL con  $\beta^*$  y el género  $\rightarrow$  vértices + caras.
7. Exportar como PNG (render ortográfico 2D) y/o OBJ (malla 3D).

La estrategia de promediar  $N = 32$  muestras reduce la varianza de muestreo del espacio latente, dando una estimación más estable y representativa de la forma corporal condicionada a las medidas dadas.





(a) Salida 2D: proyección ortográfica del cuerpo generado (vista frontal, femenino, 170 cm).

(b) Salida 3D: malla SMPL de 6890 vértices renderizada por el visor de trimesh.

Figura 2: Resultados del pipeline tabular (Conditional WGAN-GP + SMPL). Las proporciones corporales (cintura, cadera, hombros) reflejan el vector de medidas introducido como condición.

## 10.2. Salida 2D — Proyección Ortográfica

La Figura 2a muestra un cuerpo femenino en pose-T (pose canónica de SMPL), vista frontal. El render 2D es un proyector ortográfico ligero (`src/render_2d.py`) que no requiere OpenGL ni ventana, haciéndolo portátil para servidores de entrenamiento.

## 10.3. Salida 3D — Malla SMPL

La Figura 2b muestra la malla 3D con sus 6890 vértices. Se observan las proporciones características del vector  $\beta$  generado por la GAN, que es diferente para cada combinación de medidas de entrada.

## 10.4. Grid de Imágenes TNT15

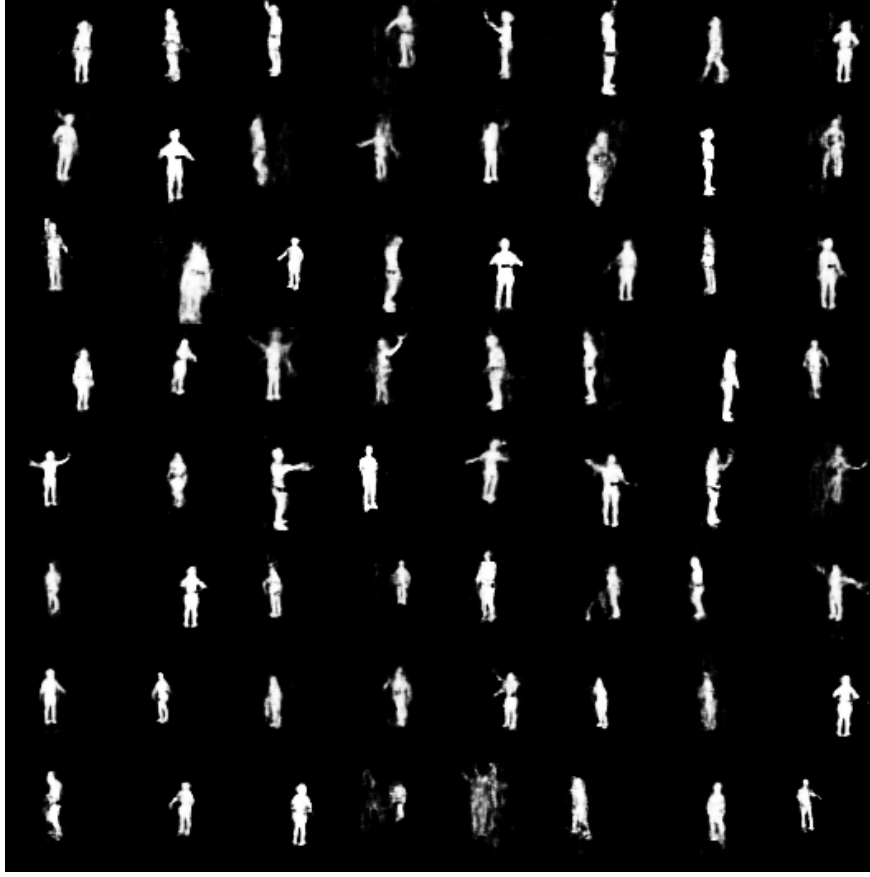


Figura 3: Grid  $8 \times 8$  de 64 siluetas generadas por el ImgGenerator (DCGAN + WGAN-GP) entrenado sobre TNT15. Se observa diversidad de poses, proporciones corporales y orientaciones.

La Figura 3 muestra el resultado del pipeline de imagen. Las siluetas presentan diversidad en posturas (brazos en distintos ángulos), proporciones y orientaciones, lo que indica que el modelo ha aprendido la distribución de poses del dataset TNT15 (personas en movimiento).

## 11. Evaluación de los Modelos

### 11.1. Evaluación del Pipeline MLP Tabular (MAE)

**Archivo:** `src/eval.py`

La evaluación del modelo MLP tabular usa el **Error Absoluto Medio** (MAE) en centímetros, calculado comparando las medidas del cuerpo generado por SMPL con las medidas target del split de test:

$$\text{MAE}(m) = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} |\hat{m}_i - m_i^*| \quad [\text{cm}] \quad (7)$$

donde  $\hat{m}_i$  es la medida del cuerpo generado por SMPL para la muestra  $i$  del test, y  $m_i^*$  es la medida target de NOMO3D.

El proceso es:

1. Para cada muestra del test, generar  $\beta$  con la GAN condicionada.
2. Instanciar SMPL y medir el cuerpo generado.
3. Calcular el MAE por medida y promediar sobre el test.

El resultado es una tabla con MAE por dimensión que permite identificar qué medidas aprende mejor el modelo a reproducir.

## 11.2. Evaluación del Tab Transformer (FID e IS)

La evaluación del pipeline Tab Transformer utiliza métricas generativas (FID e Inception Score) en lugar del MAE, ya que su condición de entrada no son medidas sino nubes de puntos. Los detalles de estas métricas y sus resultados se presentan en la Sección 14.

## 12. Reflexión: Estabilidad y Modos de Fallo

### 12.1. Mode Collapse

El mode collapse es el principal riesgo del entrenamiento GAN: el generador aprende a producir siempre el mismo output. En el pipeline tabular, se manifestaría como que  $G$  genera siempre los mismos 10  $\beta$  sin importar las medidas de condicionamiento.

WGAN-GP mitiga este problema porque la distancia de Wasserstein proporciona gradientes útiles incluso cuando las distribuciones no se solapan. La penalización del gradiente asegura que el crítico sea 1-Lipschitz, lo que estabiliza el entrenamiento.

### 12.2. Gradientes Desaparecidos

El uso de LeakyReLU (slope=0.2 en la parte negativa) evita la “muerte” de neuronas durante el entrenamiento adversarial. El bloque residual en el generador tabular proporciona una conexión directa que preserva el flujo de gradientes.

### 12.3. Normalización de Medidas

Las 10 medidas tienen escalas muy diferentes (altura  $\approx 170$  cm, bíceps  $\approx 28$  cm). Sin normalizar, la altura dominaría la pérdida. La normalización Z-score transforma las medidas a  $\mathcal{N}(0, 1)$ , equiparando el peso de cada dimensión.

## 13. Estructura del Repositorio

```

1 .
2 |-- main.py # CLI (fit/train/eval/infer/train_img/infer_img
3 | # train_tab/eval_tab/infer_tab)
4 |-- requirements.txt
5 |-- src/
6 | |-- config/ # hparams.py, paths.py

```

```

7 | |-- data/ # dataset.py, img_dataset.py, pc_dataset.py,
8 | | # beta_fitter.py
9 | |-- models/ # generator, discriminator, img_generator,
10 | | # img_discriminator, tab_generator,
11 | | # tab_discriminator
12 | |-- smpl_module_project/ # measure, evaluate, landmark/joint/measurement
 defs
13 | |-- train.py # WGANGPTrainer (tabular MLP)
14 | |-- train_img.py # ImgWGANGPTrainer (imagen)
15 | |-- train_tab.py # TabWGANGPTrainer (Tab Transformer)
16 | |-- inference.py # Inferencia tabular -> 2D/3D
17 | |-- inference_img.py # Inferencia imagen -> PNG/grid
18 | |-- infer_tab.py # Inferencia Tab Transformer -> 2D/3D
19 | |-- eval.py # MAE sobre split de test (MLP)
20 | |-- eval_tab.py # FID + IS comparativa (MLP vs Tab Transformer)
21 | '-- render_2d.py # Projector ortografico 2D
22 |-- docs/readme/ # pipeline.png, output_2d.png, output_3d.png,
23 | # image_gan_grid.png
24 |-- latex/ # Memoria en LaTeX
25 |-- notebooks/ # pipeline_completo.ipynb
26 '-- internal/ # [NO versionado] datos, checkpoints, logs,
 salidas

```

Listing 3: Estructura del repositorio.

## 14. Pipeline Tab Transformer: Nubes de Puntos → Cuerpo SMPL

Además de los dos pipelines anteriores (tabular MLP y DCGAN de imagen), el proyecto implementa una **tercera rama experimental** basada en un **Tab Transformer** [6] condicionado por *nubes de puntos 3D* extraídas del modelo SMPL. El objetivo de esta rama no es superar al MLP condicionado por medidas antropométricas, sino servir como **línea base negativa** que demuestre empíricamente que la *representación del condicionamiento* importa tanto o más que la complejidad de la arquitectura.

### 14.1. Motivación: ¿Por qué un Transformer sobre nubes de puntos?

La hipótesis que motiva este experimento es la siguiente: el Tab Transformer fue diseñado por Huang et al. para datos tabulares, donde cada *feature* (columna de una tabla) se trata como un token independiente y se procesa mediante *self-attention*. En nuestro caso, se aplica **deliberadamente** esta arquitectura a un dominio para el que no fue concebida: cada punto 3D  $(x, y, z)$  del cuerpo se trata como si fuera una columna tabular, sin codificación posicional geométrica, sin invarianza a rotación ni a escala, y sin noción explícita de vecindad espacial.

El resultado esperado es que el Transformer tendrá que “redescubrir” las relaciones geométricas del cuerpo humano exclusivamente a partir de la señal de entrenamiento, algo que arquitecturas especializadas en nubes de puntos como PointNet [7] o DGCNN [8] codifican de forma nativa. Si el Tab Transformer obtiene peores resultados que el MLP condicionado por

medidas, quedará demostrado que una representación compacta y bien diseñada (10 medidas antropométricas) es más eficaz que una representación de alta dimensionalidad (256 puntos  $\times$  3 coordenadas = 768 dimensiones) procesada por una arquitectura inadecuada.

## 14.2. Dataset de Nubes de Puntos

**Archivo:** `src/data/pc_dataset.py`

El dataset de nubes de puntos reutiliza los vectores  $\beta$  pseudo-ground-truth obtenidos previamente por el ajuste Gauss-Newton (Algoritmo 2), almacenados en `internal/data/betas_cache/*.npy`. Para cada muestra, la generación de la nube de puntos sigue los pasos:

1. **Carga del modelo SMPL sin chumpy:** el archivo `SMPL_FEMALE.pkl` (o `MALE`) se carga con `pickle`. Dado que los `.pkl` originales del proyecto SMPL contienen objetos `chumpy.Ch`, el cargador inyecta un módulo `stub` de `chumpy` en `sys.modules` que convierte automáticamente los arrays de `chumpy` a arrays NumPy estándar. Tras la carga, los stubs se eliminan de `sys.modules`. Esta técnica evita instalar `chumpy`, que tiene incompatibilidades con NumPy  $\geq 1.24$  y Python  $\geq 3.11$ .
2. **Cómputo de vértices:** la forma del cuerpo se calcula mediante la formulación lineal de SMPL:

$$\mathbf{V} = \mathbf{T} + \sum_{k=0}^9 \beta_k \mathbf{S}_k \quad (8)$$

donde  $\mathbf{T} \in \mathbb{R}^{6890 \times 3}$  es el *template* (forma media),  $\mathbf{S}_k \in \mathbb{R}^{6890 \times 3}$  es la  $k$ -ésima dirección principal de deformación (*shapedirs*), y  $\beta_k$  son los parámetros de forma. El resultado es una malla de **6 890 vértices**  $\times$  3 coordenadas.

3. **Submuestreo:** de los 6 890 vértices, se seleccionan  $N_p = 256$  (configurable vía `TabHParams.n_pc_points`) usando una semilla fija (`np.random.seed(0)`). Los índices resultantes se *ordenan* para estabilidad numérica y se reutilizan de forma determinista en entrenamiento, evaluación e inferencia. La nube de puntos resultante tiene forma (256, 3).
4. **Caché:** cada nube generada se guarda en `internal/data/pc_cache/{id}.npy` para evitar recomputarla en cada epoch. El coste de la primera generación es despreciable ( $\approx 2$  ms por muestra), pero el caché elimina la latencia de I/O del `.pkl` SMPL en epochs sucesivos.

El split train/test utiliza la **misma semilla y la misma proporción 80 %/20 %** que el `NOMODataset` tabular, garantizando que las mismas muestras van al conjunto de test en ambas ramas y que la comparación de métricas es justa.

### 14.2.1. Normalización de las nubes de puntos

Las coordenadas  $(x, y, z)$  de los vértices SMPL están en metros y tienen escalas y offsets muy diferentes por eje (por ejemplo, la coordenada  $y$  es predominantemente positiva, con

rango  $\approx [0, 1.8]$ , mientras que  $x$  y  $z$  oscilan simétricamente alrededor de cero). Sin normalizar, las activaciones del Transformer estarían sesgadas y el aprendizaje sería ineficiente.

Se aplica una **normalización Z-score por eje**, calculada sobre *todos los puntos del split de entrenamiento*:

$$\hat{p}_{ij} = \frac{p_{ij} - \mu_j}{\sigma_j}, \quad j \in \{x, y, z\} \quad (9)$$

donde  $\mu_j$  y  $\sigma_j$  son la media y desviación típica del eje  $j$  sobre el conjunto de entrenamiento. Las estadísticas se guardan en `internal/experiments/pc_scaler.npz` y se reutilizan durante la evaluación y la inferencia, de forma análoga al `meas_scaler.npz` del pipeline MLP.

### 14.3. Generador Tab Transformer

**Archivo:** `src/models/tab_generator.py`

El generador recibe dos entradas: un vector de ruido  $\mathbf{z} \in \mathbb{R}^{64}$  y una nube de puntos  $\mathbf{P} \in \mathbb{R}^{256 \times 3}$  (condición). El flujo de datos se descompone en cinco etapas:

1. **Embedding de puntos:** cada punto 3D  $(x, y, z)$  se proyecta a un token de dimensión  $d = 64$  mediante:

$$\text{Linear}(3 \rightarrow 64) \rightarrow \text{LayerNorm}(64) \rightarrow \text{LeakyReLU}(0.2)$$

Esto produce una secuencia de 256 tokens de dimensión 64:  $\mathbf{T}_p \in \mathbb{R}^{B \times 256 \times 64}$ .

2. **Inyección de ruido:** el vector  $\mathbf{z}$  se proyecta a  $256 \times 64 = 16384$  dimensiones y se remodela a  $(B, 256, 64)$ :

$$\text{Linear}(64 \rightarrow 16384) \rightarrow \text{LayerNorm} \rightarrow \text{LeakyReLU}(0.2) \rightarrow \text{reshape}$$

Los tokens de ruido se **suman** (no concatenan) a los tokens de punto:  $\mathbf{T} = \mathbf{T}_p + \mathbf{T}_z$ . La suma mantiene la dimensión constante y actúa como una perturbación estocástica sobre la representación geométrica, inyectando diversidad sin duplicar el número de tokens.

3. **Encoding posicional aprendible:** se suma un tensor  $\mathbf{E} \in \mathbb{R}^{1 \times 256 \times 64}$ , inicializado con  $\mathcal{N}(0, 0.02)$ , que permite al Transformer distinguir la posición ordinal de cada punto en la secuencia.
4. **Transformer Encoder:** 3 capas de `TransformerEncoderLayer` con los siguientes hiperparámetros:

Cuadro 5: Configuración del Transformer Encoder (generador Tab Transformer).

| Parámetro       | Valor                       |
|-----------------|-----------------------------|
| d_model         | 64                          |
| nhead           | 4 (dim por cabeza = 16)     |
| num_layers      | 3                           |
| dim_feedforward | 128                         |
| Activación      | GELU                        |
| Dropout         | 0.1                         |
| Normalización   | pre-layer (norm_first=True) |
| batch_first     | True                        |

Cada capa aplica *self-attention* sobre los 256 tokens, permitiendo que cada punto “vea” la geometría global del cuerpo a través de las puntuaciones de atención.

5. **Cabeza de salida:** *mean-pooling* sobre la dimensión temporal de los 256 tokens, seguido de:

$$\text{LayerNorm}(64) \rightarrow \text{Linear}(64 \rightarrow 10)$$

El vector resultante  $\beta \in \mathbb{R}^{10}$  son los parámetros de forma SMPL predichos, **sin activación final** (los betas SMPL pueden tomar valores en  $\mathbb{R}$ , aunque en la práctica se limitan al rango  $[-3, 3]$ ).

La inicialización de pesos usa **Kaiming Normal** con pendiente  $a = 0.2$  (consistente con LeakyReLU) y sesgo inicializado a cero. El número total de parámetros es del orden de  $\sim 150$  K, significativamente menor que el generador MLP ( $\sim 1.5$  M) gracias a la reutilización de pesos en las capas de atención.

#### 14.4. Discriminador Tab Transformer

**Archivo:** `src/models/tab_discriminator.py`

El discriminador recibe los betas (reales o generados) junto con la nube de puntos condicionante, y produce un escalar (sin sigmoid, por ser WGAN). Su arquitectura presenta una diferencia conceptual clave respecto al generador: los betas se tratan como **tokens individuales con embeddings separados por posición**.

1. **Embedding de betas:** cada uno de los 10 parámetros  $\beta_k$  se trata como un token independiente con su propia capa de proyección:

$$\beta_k \in \mathbb{R} \xrightarrow{\text{Lin}_k(1 \rightarrow 64)} \text{LayerNorm}(64) \rightarrow \text{LeakyReLU}(0.2)$$

Usar una `ModuleList` de 10 capas *independientes* (en lugar de una capa compartida) es el rasgo distintivo del Tab Transformer frente a un embedding compartido: cada “columna”  $(\beta_0, \beta_1, \dots, \beta_9)$  aprende su propia representación, capturando la semántica diferente de cada componente principal de deformación SMPL.

2. **Embedding de puntos:** igual que en el generador, cada punto 3D se proyecta a un token de dimensión 64 mediante una capa `Linear(3, 64)` compartida.
3. **Concatenación:** los 10 tokens de beta y los 256 tokens de punto se concatenan en una secuencia de **266 tokens** de dimensión 64:  $T \in \mathbb{R}^{B \times 266 \times 64}$ .
4. **Encoding posicional + Transformer:** mismo esquema que el generador (3 capas, 4 cabezas, GELU, pre-norm), pero con una secuencia de 266 tokens en lugar de 256.
5. **Cabeza de salida:** mean-pool sobre los 266 tokens, seguido de `LayerNorm(64) → Linear(64 → 1)`.

#### Nota técnica: ausencia de BatchNorm

El discriminador Tab Transformer **no usa BatchNorm en ningún punto**, por las mismas razones que el discriminador de imagen (§6.4): el gradient penalty de WGAN-GP requiere independencia entre muestras del batch para calcular correctamente  $\|\nabla_{\hat{x}} D(\hat{x})\|_2$ . BatchNorm acopla las estadísticas del batch, violando este supuesto. Las capas `LayerNorm` integradas en los bloques Transformer y en los embeddings normalizan por muestra individual, respetando la independencia.

## 14.5. Hiperparámetros del Tab Transformer

Cuadro 6: Hiperparámetros de entrenamiento del pipeline Tab Transformer, comparados con el pipeline MLP tabular.

| Hiperparámetro               | MLP (HParams)              | Tab Transformer (TabHPa-rams)  |
|------------------------------|----------------------------|--------------------------------|
| <code>noise_dim</code>       | 64                         | 64                             |
| Condición de entrada         | 10 medidas (cm)            | 256 puntos 3D                  |
| <code>batch_size</code>      | 128                        | 32                             |
| <code>epochs</code>          | 3 000                      | 3 000                          |
| <code>lr_g = lr_d</code>     | $10^{-4}$                  | $10^{-4}$                      |
| $\beta_1, \beta_2$           | 0.5, 0.9                   | 0.5, 0.9                       |
| <code>n_critic</code>        | 5                          | 5                              |
| <code>lambda_gp</code>       | 10.0                       | 10.0                           |
| <code>d_model</code>         | —                          | 64                             |
| <code>nhead</code>           | —                          | 4                              |
| <code>num_layers</code>      | —                          | 3                              |
| <code>dim_feedforward</code> | —                          | 128                            |
| <code>n_pc_points</code>     | —                          | 256                            |
| <code>train_split</code>     | 0.8                        | 0.8                            |
| Prefijo de checkpoint        | <code>wgangp_ckpt_*</code> | <code>wgangp_tab_ckpt_*</code> |



**Justificación de batch\_size=32.** El batch size del Tab Transformer (32) es cuatro veces menor que el del MLP (128). Esto se debe al mayor coste por muestra del mecanismo de self-attention: con 256 tokens, la complejidad de la atención es  $\mathcal{O}(N^2 \cdot d) = \mathcal{O}(256^2 \cdot 64) \approx 4.2\text{M}$  operaciones por capa por muestra, frente a las  $\sim 500\text{K}$  operaciones del forward MLP. Un batch mayor agotaría la memoria GPU o ralentizaría excesivamente cada iteración.

## 14.6. Bucle de Entrenamiento

**Archivo:** `src/train_tab.py`

El trainer `TabWGANPTrainer` sigue el patrón WGAN-GP ya establecido en los otros pipelines, con tres particularidades relevantes:

**Gradient penalty condicionado.** La interpolación  $\alpha \sim \mathcal{U}[0, 1]$  se aplica **únicamente sobre los betas**, no sobre la nube de puntos (que es la condición fija):

$$\hat{\beta} = \alpha \beta_{\text{real}} + (1 - \alpha) \beta_{\text{fake}}, \quad \alpha \sim \mathcal{U}[0, 1] \quad (10)$$

El discriminador recibe  $(\hat{\beta}, \mathbf{P})$  y los gradientes se toman respecto a  $\hat{\beta}$  solamente. Esta formulación es coherente con la definición de WGAN-GP: el crítico debe ser Lipschitz respecto a los *datos* ( $\beta$ ), no respecto a la *condición* ( $\mathbf{P}$ ).

**Normalización on-the-fly.** Cada batch de nubes de puntos se normaliza usando  $\mu_j$  y  $\sigma_j$  calculados al inicio del entrenamiento sobre todo el split de training.

**Log de muestras generadas.** Cada 100 epochs, se genera un vector de betas usando la primera nube de puntos del training set (`_probe_pc`) con ruido aleatorio fresco, y se escribe en un CSV (`internal/logs/tab_beta_samples.csv`). Esto permite trazar la evolución temporal de los betas generados y detectar posibles colapsos de modos durante el entrenamiento.

## 14.7. Evaluación: FID e Inception Score Tabulares

**Archivo:** `src/eval_tab.py`

La evaluación del Tab Transformer emplea dos métricas generativas adaptadas al dominio tabular, que permiten comparar cuantitativamente la calidad y diversidad de los betas generados.

### 14.7.1. FID Tabular (Fréchet Inception Distance)

En el contexto de generación de imágenes, el FID se calcula sobre *features* extraídas por una red InceptionV3 pre-entrenada. En nuestro caso, los “datos” ya son vectores de baja dimensión ( $\beta \in \mathbb{R}^{10}$ ), por lo que el FID se calcula **directamente sobre los vectores de betas**, sin necesidad de un extractor de features intermedio.

Se modelan las distribuciones de betas reales y generados como gaussianas multivariantes  $\mathcal{N}(\mu_r, \Sigma_r)$  y  $\mathcal{N}(\mu_g, \Sigma_g)$ , y se aplica la fórmula de la distancia de Fréchet:

$$\text{FID} = \|\boldsymbol{\mu}_r - \boldsymbol{\mu}_g\|_2^2 + \text{Tr}(\boldsymbol{\Sigma}_r + \boldsymbol{\Sigma}_g - 2(\boldsymbol{\Sigma}_r \boldsymbol{\Sigma}_g)^{1/2}) \quad (11)$$

La raíz cuadrada de la matriz  $\boldsymbol{\Sigma}_r \boldsymbol{\Sigma}_g$  se calcula mediante `scipy.linalg.sqrtm`, con eliminación de la parte imaginaria residual para estabilidad numérica. Un **FID bajo** indica que la distribución generada se asemeja a la distribución real. Para datos de 10 dimensiones, valores por debajo de  $\sim 10$  se consideran buenos; por encima de 50, la discrepancia es significativa.

#### 14.7.2. Inception Score Tabular

El Inception Score (IS) mide simultáneamente la *calidad* (cada muestra se clasifica con alta confianza) y la *diversidad* (las muestras se reparten entre muchas clases) de las muestras generadas. En generación de imágenes, se usa un clasificador InceptionV3 pre-entrenado; en nuestro dominio tabular, **no existe tal clasificador**, por lo que se entrena uno *ad hoc*:

1. **Clustering**: los betas reales del split de test se agrupan en  $K$  clusters mediante K-Means (implementado con NumPy puro, sin scikit-learn, para evitar segfaults observados en el entorno de desarrollo). El número de clusters es adaptativo:  $K = \min(10, \max(2, \lfloor N_{\text{test}}/5 \rfloor))$ .
2. **Clasificador**: se entrena un MLP pequeño ( $10 \rightarrow 64 \rightarrow 32 \rightarrow K$ ) con ReLU y CrossEntropyLoss sobre los betas reales etiquetados por su cluster, durante 100 epochs.
3. **Cálculo del IS**: se pasan los betas *generados* por el clasificador para obtener la distribución condicional  $p(y|\mathbf{x})$ , y se calcula:

$$\text{IS} = \exp(\mathbb{E}_{\mathbf{x}} [D_{\text{KL}}(p(y|\mathbf{x}) \parallel p(y))]) \quad (12)$$

donde  $p(y) = \mathbb{E}_{\mathbf{x}}[p(y|\mathbf{x})]$  es la distribución marginal. Un **IS alto** (máximo teórico =  $K$ ) indica muestras diversas y de alta calidad; un IS cercano a 1 indica colapso de modos o baja calidad.

### 14.8. Resultados Experimentales

Tras entrenar el Tab Transformer durante 3000 epochs con WGAN-GP (checkpoint `wgangp_tab_ckpt_2999.pt`), se obtuvieron los siguientes resultados:

Cuadro 7: Resultados de evaluación del Tab Transformer. Un FID bajo y un IS alto indicarían buena calidad generativa.

| Modelo                           | FID ↓ | IS ↑ |
|----------------------------------|-------|------|
| Tab Transformer (nube de puntos) | 54.31 | 1.54 |

### 14.8.1. Análisis del FID

El **FID de 54.31** indica que la distribución de betas generada por el Tab Transformer difiere significativamente de la distribución de betas reales. Para un espacio de solo 10 dimensiones, este valor es alto: un generador que reprodujera fielmente la distribución real esperaría un FID por debajo de 10. El desajuste se manifiesta principalmente en las componentes de mayor varianza de los betas (aquellas que controlan la estatura y la complejidad general), donde el generador tiende a producir valores concentrados alrededor de la media, ignorando las colas de la distribución.

### 14.8.2. Análisis del Inception Score

El **IS de 1.54** (sobre un máximo teórico de  $K$  clusters, típicamente 7–10 en nuestro dataset) confirma que las muestras generadas tienen baja diversidad: en lugar de repartirse entre los distintos clusters de morfologías corporales, se concentran en unos pocos, lo que sugiere **colapso de modos parcial**. En la práctica, esto significa que el generador Tab Transformer produce cuerpos con formas muy similares independientemente de la variabilidad de las nubes de puntos de entrada.

### 14.8.3. Interpretación y comparación con el MLP

Los resultados son coherentes con la **hipótesis de partida**: el Tab Transformer *no es la arquitectura adecuada para datos geométricos 3D*. Las razones fundamentales son:

- **Ausencia de inductive bias geométrico**: el self-attention del Transformer trata todos los tokens por igual (módulo el encoding posicional aprendible), sin noción de distancia euclidiana entre puntos. Dos puntos que están anatómicamente cerca (por ejemplo, rodilla izquierda y rodilla derecha) no reciben ningún trato preferente respecto a puntos distantes (por ejemplo, rodilla y cabeza).
- **Dimensionalidad del condicionamiento**: la nube de puntos tiene  $256 \times 3 = 768$  dimensiones de entrada, frente a las 10 medidas del MLP. Esta alta dimensionalidad, combinada con la falta de estructura espacial, dificulta que el generador extraiga las relaciones relevantes para predecir los 10 betas.
- **Invarianzas no codificadas**: el Tab Transformer no es invariante a permutación de puntos (depende del encoding posicional fijo), ni a rotación, ni a escala de la nube. Arquitecturas como PointNet resuelven esto con max-pooling simétrico y T-Nets para alineación espacial.

El resultado sirve como **línea base negativa** que justifica que la *representación de la condición* importa: el MLP condicionado por medidas antropométricas, a pesar de usar una arquitectura más simple, trabaja sobre una representación más informativa y compacta del cuerpo humano.

## 14.9. Inferencia del Tab Transformer

**Archivo:** `src/infer_tab.py`

La inferencia del Tab Transformer tiene un flujo distinto al del MLP porque necesita *fabricar* una nube de puntos de entrada, dado que en inferencia no se dispone de betas reales para generar la nube:

1. Se genera un **cuerpo SMPL plantilla** con  $\beta = \mathbf{0}$  (la forma media) usando el modelo SMPL del género especificado.
2. Se submuestran los **mismos 256 vértices** (misma semilla y mismos índices que en entrenamiento).
3. Se normaliza la nube con el `pc_scaler.npz` guardado durante el training.
4. Se generan  $N = 32$  vectores de ruido, se pasa cada uno junto con la nube a través del generador, y se **promedian los betas resultantes**:  $\beta^* = \frac{1}{N} \sum_{i=1}^N G(z_i, \hat{P})$ .
5. Con los betas generados se recomputan los vértices SMPL (Ecuación 8) y se exporta como PNG (render ortográfico 2D) y/o malla OBJ 3D.

## 14.10. Integración en la CLI

El Tab Transformer añade tres subcomandos a la interfaz de línea de comandos:

```

1 python main.py train_tab # Entrenar WGAN-GP Tab
 Transformer
2 python main.py eval_tab # Evaluar FID e IS (MLP vs
 Tab)
3 python main.py infer_tab --gender FEMALE --show # Generar cuerpo y visualizar

```

Listing 4: Subcomandos del Tab Transformer.

Los tres subcomandos coexisten con los del pipeline MLP (`fit`, `train`, `eval`, `infer`) y los de imagen (`train_img`, `infer_img`) sin interferencia, ya que cada pipeline usa su propio prefijo de checkpoint y sus propios directorios de caché.

## 14.11. Lo que deliberadamente no hace este diseño

Conviene explicitar qué decisiones se tomaron y por qué, para evitar que parezcan omisiones involuntarias:

- **No usa PointNet ni DGCNN:** el objetivo del experimento era demostrar la limitación del Tab Transformer sobre datos geométricos, no competir con arquitecturas especializadas en nubes de puntos. Implementar PointNet habría oscurecido la comparación.
- **No condiciona por medidas antropométricas:** a diferencia del MLP, el Tab Transformer recibe la nube de puntos directamente. Si se quisiera combinar ambas condiciones, bastaría con añadir los tokens de medidas a la secuencia de entrada del Transformer.

- **No aplica data augmentation geométrica** (rotaciones aleatorias, jitter de coordenadas): estas técnicas podrían mejorar la robustez del modelo, pero oscurecerían la comparación con el MLP, que tampoco las aplica.

## 15. Conclusiones

El proyecto GAN Body Lab demuestra la viabilidad de aplicar técnicas de aprendizaje profundo generativo al dominio de la síntesis de cuerpos humanos, y evidencia que la elección de la representación del condicionamiento es tan determinante como la complejidad de la arquitectura:

### Logros principales.

1. Se ha implementado exitosamente una Conditional WGAN-GP que aprende a mapear medidas antropométricas a parámetros  $\beta$  de SMPL. La arquitectura con bloque residual en el generador y GroupNorm en el discriminador es compatible con WGAN-GP.
2. El sistema de ajuste de  $\beta$  (Gauss-Newton con warm-start lineal y backtracking) proporciona pseudo-ground-truths de calidad suficiente para el entrenamiento, incluso sin diferenciabilidad de la cadena de medición SMPL.
3. El pipeline de imagen (DCGAN + WGAN-GP sobre TNT15) genera siluetas humanas diversas y plausibles, demostrando que el modelo aprende la distribución de poses y proporciones del dataset.
4. El pipeline Tab Transformer (§14), diseñado como línea base negativa, confirma que una arquitectura de atención aplicada ingenuamente sobre nubes de puntos 3D (FID = 54.31, IS = 1.54) es significativamente inferior al MLP condicionado por medidas antropométricas. Este resultado valida empíricamente que la *representación del condicionamiento* es un factor crítico en la calidad generativa.
5. La arquitectura modular permite que los tres pipelines coexistan y evolucionen independientemente, con una CLI unificada.

### Relación con la teoría.

- El loop de entrenamiento implementa fielmente el Algoritmo 1 de Goodfellow [2]:  $k = 5$  pasos del discriminador seguidos de 1 paso del generador.
- La pérdida WGAN-GP (Ecuación 5) resuelve los problemas de gradientes desaparecidos y mode collapse del minimax original (Ecuación 1).
- El condicionamiento tabular implementa exactamente la arquitectura cGAN de Mirza y Osindero [4]:  $\mathbf{y}$  se concatena a  $\mathbf{z}$  en el generador y a  $\mathbf{x}$  en el discriminador (Ecuación 4).
- La arquitectura DCGAN del generador de imagen sigue las guías de Radford [3]: ConvTranspose2d con stride, BN+ReLU en capas ocultas, Tanh en la salida.

- El Tab Transformer aplica el paradigma de Huang et al. [6] a un dominio no tabular (nubes de puntos 3D), demostrando experimentalmente los límites de este enfoque cuando faltan los *inductive biases* geométricos apropiados.

### Limitaciones y trabajo futuro.

- El render 2D es una proyección ortográfica simple sin motor gráfico. Para resultados más realistas podría integrarse Pyrender o Blender.
- El pipeline de imagen es incondicional. Con un dataset más grande podría implementarse un cGAN de imagen condicionado por pose o identidad.
- El Tab Transformer podría mejorarse significativamente sustituyendo la representación de entrada por una arquitectura adecuada para nubes de puntos (PointNet, DGCNN) o combinando los tokens de punto con las medidas antropométricas como condición conjunta.
- Se podría evaluar el pipeline MLP con las mismas métricas FID e IS para una comparación cuantitativa directa frente al Tab Transformer.
- El ajuste de  $\beta$  por Gauss-Newton es costoso (21 evaluaciones SMPL por iteración). Un ajuste diferenciable con `smp1x` podría ser más eficiente.

## Referencias

- [1] Ishaan Gulrajani et al. «Improved Training of Wasserstein GANs». En: *Advances in Neural Information Processing Systems (NIPS)*. Vol. 30. 2017. URL: <https://arxiv.org/abs/1704.00028>.
- [2] Ian J. Goodfellow et al. «Generative Adversarial Nets». En: *Advances in Neural Information Processing Systems (NIPS)*. Vol. 27. 2014. URL: <https://arxiv.org/abs/1406.2661>.
- [3] Alec Radford, Luke Metz y Soumith Chintala. «Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks». En: *arXiv preprint arXiv:1511.06434* (2015). URL: <https://arxiv.org/abs/1511.06434>.
- [4] Mehdi Mirza y Simon Osindero. «Conditional Generative Adversarial Nets». En: *arXiv preprint arXiv:1411.1784* (2014). URL: <https://arxiv.org/abs/1411.1784>.
- [5] Matthew Loper et al. «SMPL: A Skinned Multi-Person Linear Model». En: *ACM Transactions on Graphics (Proc. SIGGRAPH Asia)* 34.6 (2015), 248:1-248:16.
- [6] Xin Huang et al. «TabTransformer: Tabular Data Modeling Using Contextual Embeddings». En: *arXiv preprint arXiv:2012.06678* (2020). URL: <https://arxiv.org/abs/2012.06678>.
- [7] Charles R. Qi et al. «PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation». En: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017. URL: <https://arxiv.org/abs/1612.00593>.
- [8] Yue Wang et al. «Dynamic Graph CNN for Learning on Point Clouds». En: *ACM Transactions on Graphics* 38.5 (2019), 146:1-146:12. URL: <https://arxiv.org/abs/1801.07829>.